SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Computer Vision with OpenCV
COURSE CODE: DSA02

Name: N Yugesh

CO5: Apply computer vision techniques to real-world applications such as face recognition, tracking, medical imaging, and autonomous systems. (BL3)

Pseudocode Writing Task

Code of Conduct:
I N Yugesh(192424329) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Pseudocode Writing Task

1. Face Detection and Recognition Using Eigenfaces

Scenario: *Design an algorithm to detect faces in an image and recognize them using the Eigenfaces method.*

Task:

A. Input: A grayscale image (size M x N)

B. Output: Bounding boxes of detected faces and corresponding recognized names

C. Requirements: Handle multiple faces in the same image and varying lighting conditions

Pseudocode

```
ALGORITHM EigenfaceFaceDetectionRecognition
INPUT:  I            // grayscale image, size M x N
        DB           // database of known faces: list of (omega_i, name_i)
        Psi          // mean face vector (from Eigenspace training)
        U[1..k]      // top-k eigenvectors (eigenfaces)
        THRESHOLD    // max Euclidean distance to accept a match
OUTPUT: results      // list of (boundingBox, name) for every detected face

BEGIN
    // ---- Step 1: Preprocess the image ----
    I_norm   <- NormalizeIntensity(I)             // rescale pixel values to [0,255]
    I_eq     <- HistogramEqualization(I_norm)      // handle varying lighting conditions
    I_ready  <- GaussianBlur(I_eq, 3x3)            // light denoising

    // ---- Step 2: Detect faces (multi-scale sliding window / Viola-Jones) ----
    candidateBoxes <- EmptyList()
    FOR scale IN GenerateScales(minSize, maxSize, scaleFactor):
        FOR (x, y) IN SlideWindow(I_ready, scale, stride):
```

```
            window <- ExtractPatch(I_ready, x, y, scale)
            IF ViolaJonesCascade(window) == FACE THEN
                candidateBoxes.Add(BoundingBox(x, y, scale))
            END IF
        END FOR
    END FOR
    faceBoxes <- NonMaximumSuppression(candidateBoxes, iouThreshold = 0.3)
        // NMS merges overlapping detections -> correctly handles MULTIPLE faces

    // ---- Step 3 & 4: Extract Eigenface features and recognize, per face ----
    results <- EmptyList()
    FOR EACH box IN faceBoxes:
        face      <- CropAndResize(I_ready, box, stdSize)      // e.g. 100 x 100
        x_vec     <- Flatten(face)                             // 1-D pixel vector
        x_centred <- x_vec - Psi                                // mean-subtract
        omega     <- [ Dot(x_centred, U[1]), ..., Dot(x_centred, U[k]) ]
                        // omega = projection (weights) onto eigenspace

        bestName <- "Unknown"
        minDist  <- INFINITY
        FOR EACH (omega_db, name_db) IN DB:
            d <- EuclideanDistance(omega, omega_db)
            IF d < minDist THEN
                minDist  <- d
                bestName <- name_db
            END IF
        END FOR

        IF minDist >= THRESHOLD THEN
            bestName <- "Unknown"                               // reject weak matches
        END IF

        results.Add( (box, bestName) )
    END FOR

    // ---- Step 5: Output ----
    RETURN results   // bounding boxes + recognized labels
END
```

Design Notes

- Multiple faces: handled by scanning the whole image with a multi-scale sliding window / Viola-Jones cascade and running non-maximum suppression, so every face in the frame produces its own bounding box, and Steps 3-4 (Eigenface projection + matching) run independently per detected face.
- Varying lighting: handled up front in Step 1 (intensity normalization + histogram equalization) before any detection or feature extraction, so both the detector and the Eigenface projection operate on a lighting-normalized image rather than raw pixel intensities.
- Open-set recognition: a THRESHOLD on the Euclidean distance in eigenspace lets the algorithm output "Unknown" for a detected face that does not match any enrolled identity closely enough, instead of forcing an incorrect best match.
-

Expected Output (Illustration)

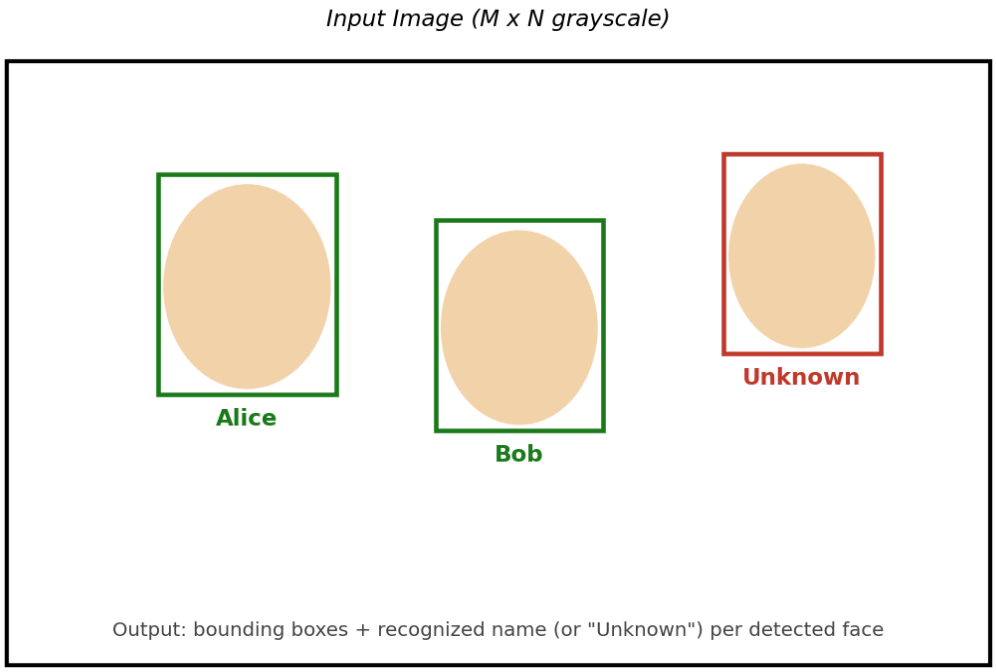

Figure 1 — Illustrative output: bounding boxes drawn around each detected face, labeled with the recognized name or "Unknown" when no database match clears the threshold.

2. Real-Time Road Lane Detection

Scenario: *Design an algorithm to detect road lanes in a real-time driving video sequence.*

Task:

A. Input: Video frames from a front-facing camera
B. Output: Lane lines highlighted on each frame
C. Requirements: Robust to shadows, curves, and partial occlusion

Pseudocode

```
ALGORITHM RealTimeLaneDetection
INPUT:  V   // live video stream from a front-facing camera
OUTPUT: V'  // same video stream with lane lines overlaid on every frame

prevLeftCurve  <- NULL     // holds last frame's fitted left-lane curve
prevRightCurve <- NULL     // holds last frame's fitted right-lane curve

BEGIN
    // ---- Step 7: repeat for every incoming frame ----
    WHILE V.HasNextFrame():

        // ---- Step 1: Capture video frame ----
        frame <- V.CaptureNextFrame()

        // ---- Step 2: Convert frame to grayscale ----
        gray <- ConvertToGrayscale(frame)

        // ---- Step 3: Apply Gaussian smoothing to reduce noise ----
```

```
        blurred <- GaussianBlur(gray, kernel = 5x5, sigma = 1.4)

        // Shadow-robustness: normalize illumination before edge detection
        blurred <- AdaptiveHistogramEqualization(blurred)   // CLAHE, softens shadow edges

        // ---- Step 4: Perform edge detection (Canny) ----
        edges <- CannyEdgeDetector(blurred, lowThresh, highThresh)

        // Restrict search to the road area (reduces false edges from shadows/scenery)
        roiMask      <- TrapezoidROI(frame.width, frame.height)
        maskedEdges  <- ApplyMask(edges, roiMask)

        // ---- Step 5: Apply Hough Transform to detect straight/curved lines ----
        lines <- HoughTransform(maskedEdges, rho = 1, theta = PI/180,
                                threshold = 50, minLineLength, maxLineGap)

        leftPts, rightPts <- ClassifyBySlopeAndPosition(lines)
                                // negative slope + left half -> left lane, etc.

        // Curve-robustness: fit a 2nd-degree polynomial, not just a straight line
        leftCurve  <- FitPolynomial(leftPts, degree = 2)
        rightCurve <- FitPolynomial(rightPts, degree = 2)

        // Occlusion-robustness: fall back to the previous frame's curve, and
        // smooth across frames, if this frame's detection is missing/weak
        IF leftCurve  == NULL OR NotEnoughPoints(leftPts)  THEN
            leftCurve <- prevLeftCurve
        ELSE
            leftCurve <- TemporalSmooth(leftCurve, prevLeftCurve, alpha = 0.8)
        END IF

        IF rightCurve == NULL OR NotEnoughPoints(rightPts) THEN
            rightCurve <- prevRightCurve
        ELSE
            rightCurve <- TemporalSmooth(rightCurve, prevRightCurve, alpha = 0.8)
        END IF

        prevLeftCurve  <- leftCurve
        prevRightCurve <- rightCurve

        // ---- Step 6: Overlay detected lanes on the original frame ----
        outFrame <- OverlayLanesOnFrame(frame, leftCurve, rightCurve)
        V'.WriteFrame(outFrame)

    END WHILE   // Step 7: loop continues until the video stream ends
    RETURN V'
END
```

Design Notes

- Shadows: an adaptive histogram equalization (CLAHE) pass normalizes local contrast before Canny edge detection, and the trapezoidal region-of-interest mask discards edges from scenery/shadow outside the road area, so shadow boundaries are much less likely to be picked up as lane edges.
- Curves: lane points are fit with a 2nd-degree polynomial rather than a single straight Hough line, so the algorithm can represent a curving lane instead of only straight segments.
- Partial occlusion: when a frame yields too few lane points (e.g., a lane briefly hidden by another vehicle or worn-out markings), the algorithm falls back to the previous frame's fitted curve, and applies temporal smoothing (weighted blend with the prior curve) even in normal frames, so a single noisy or occluded frame does not cause the overlay to jump or disappear.

- Real-time constraint: every operation in the loop (grayscale conversion, blur, Canny, Hough transform, polynomial fit) is a fast, well-optimized classical CV operation with no per-frame training, so the pipeline is lightweight enough to run frame-by-frame on live video rather than requiring batch/offline processing.

Expected Output (Illustration)

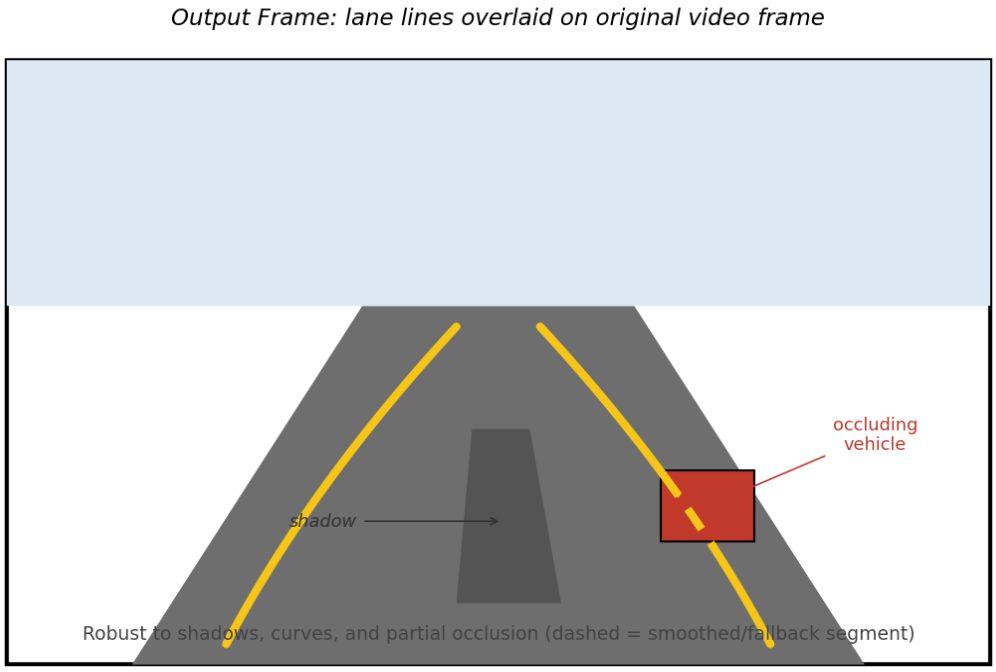

Figure 2 — Illustrative output: fitted lane curves overlaid on the frame; the dashed segment shows the smoothed/fallback curve used while the right lane is briefly occluded by a vehicle.